Rapport de TP : Validation et Analyse des Scénarios d'Erreurs d'une API REST sous Postman

1. Contexte et Objectifs du TP

L'objectif de ce TP est de concevoir et d'exécuter une suite de tests automatisés Postman pour valider le comportement d'une API REST (DummyJSON Products et Users). Une attention particulière est portée à la gestion standardisée des erreurs HTTP (codes 400 Bad Request et 404 Not Found), à l'extraction de variables d'environnement et aux mesures de temps de réponse.

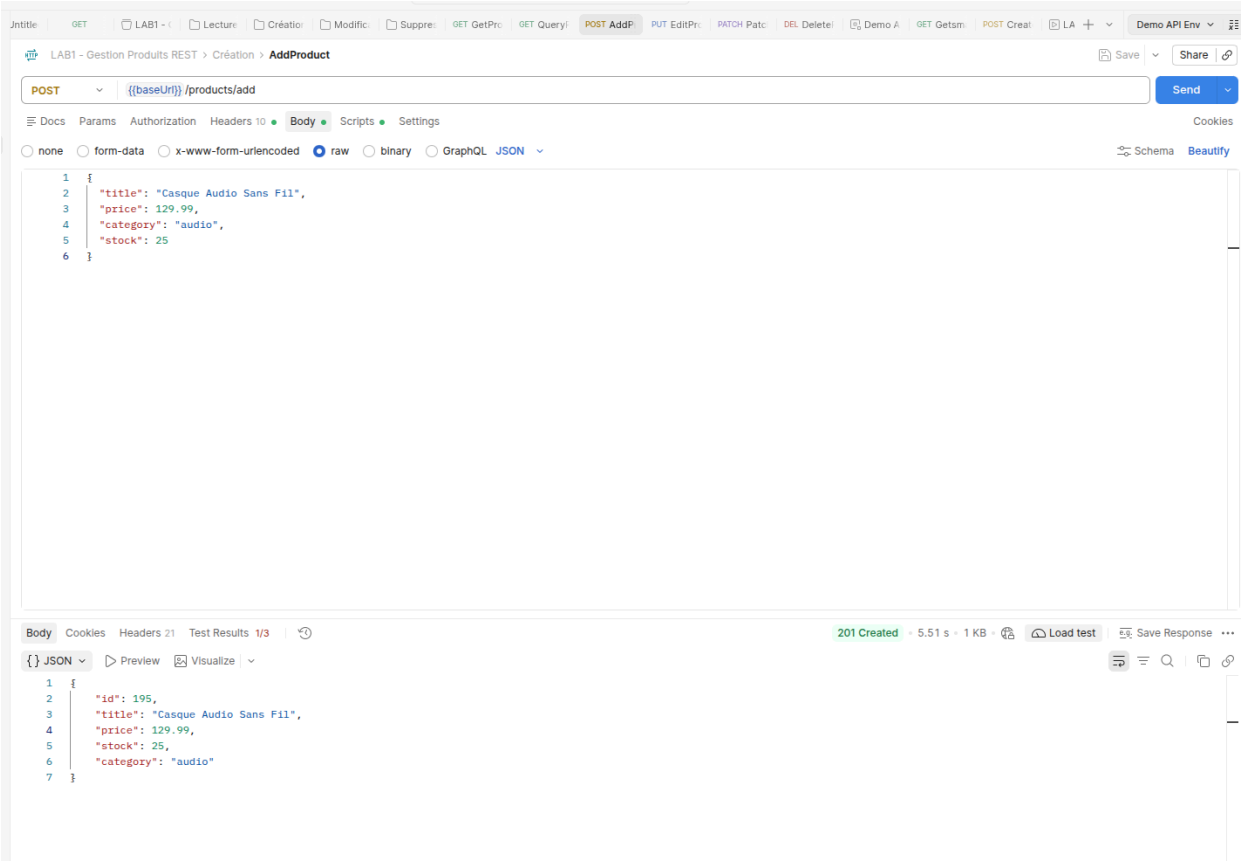
2. Synthèse des Observations et Explication des Codes HTTP

- 200 OK (Succès) : Requêtes GET (recherche, catégories, produit par ID) et modifications (PUT, PATCH). Indique que la requête a été traitée avec succès par le serveur et que les données demandées ou modifiées sont renvoyées.
- 201 Created (Créé avec succès) : Requêtes POST (AddProduct, CreateUser). Confirms la création effective d'une nouvelle ressource sur le serveur.
- 400 Bad Request (Requête incorrecte) : Envoi de données malformées ou de types d'attributs invalides dans le corps JSON. Indique un rejet par l'API dû à une erreur côté client (validation de schéma, syntaxe JSON invalide).
- 404 Not Found (Ressource introuvable) : Interrogation d'un produit avec un ID inexistant (/products/99999). Signale que le serveur ne trouve aucune ressource correspondant à l'URI fournie.

3. Détail des Requêtes et des Tests Automatisés

3.1. Dossier : Création

A. Requête AddProduct (POST {{baseUrl}}/products/add)



- Description : Ajout d'un produit ("Casque Audio Sans Fil", prix: 129.99, catégorie: "audio", stock: 25).
- Script de Test Postman :

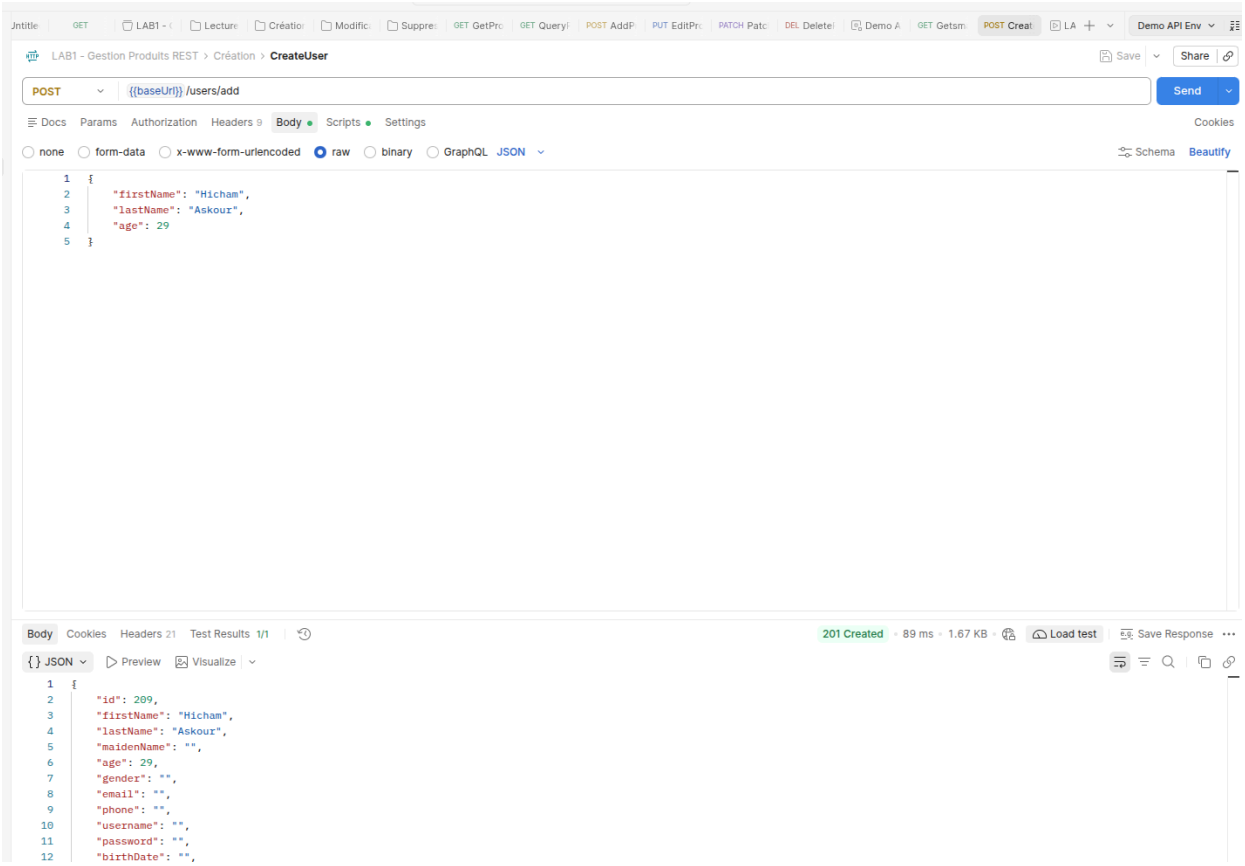
```javascript
// Vérifier que le statut de création est 201 ou 200
pm.test("Statut de création valide", function () {
    pm.response.to.have.status(201);
});

// Extraire l'ID du produit créé et le stocker dans les variables d'environnement
var jsonData = pm.response.json();
pm.environment.set("createdProductId", jsonData.id);
console.log("ID du produit créé : " + jsonData.id);
```

B. Requête CreateUser (POST {{baseUrl}}/users/add)

- Description : Création d'un profil utilisateur.
- Script de Test Postman :

```javascript
pm.test("Vérifier que le temps de réponse (Response Time) de l'API est inférieur à 500 ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(500);
});
```



3.2. Dossier : Lecture et Scénarios d'Erreurs

A. Consultation et Test d'Erreur 404 Not Found (GET {{baseUrl}}/products/99999)

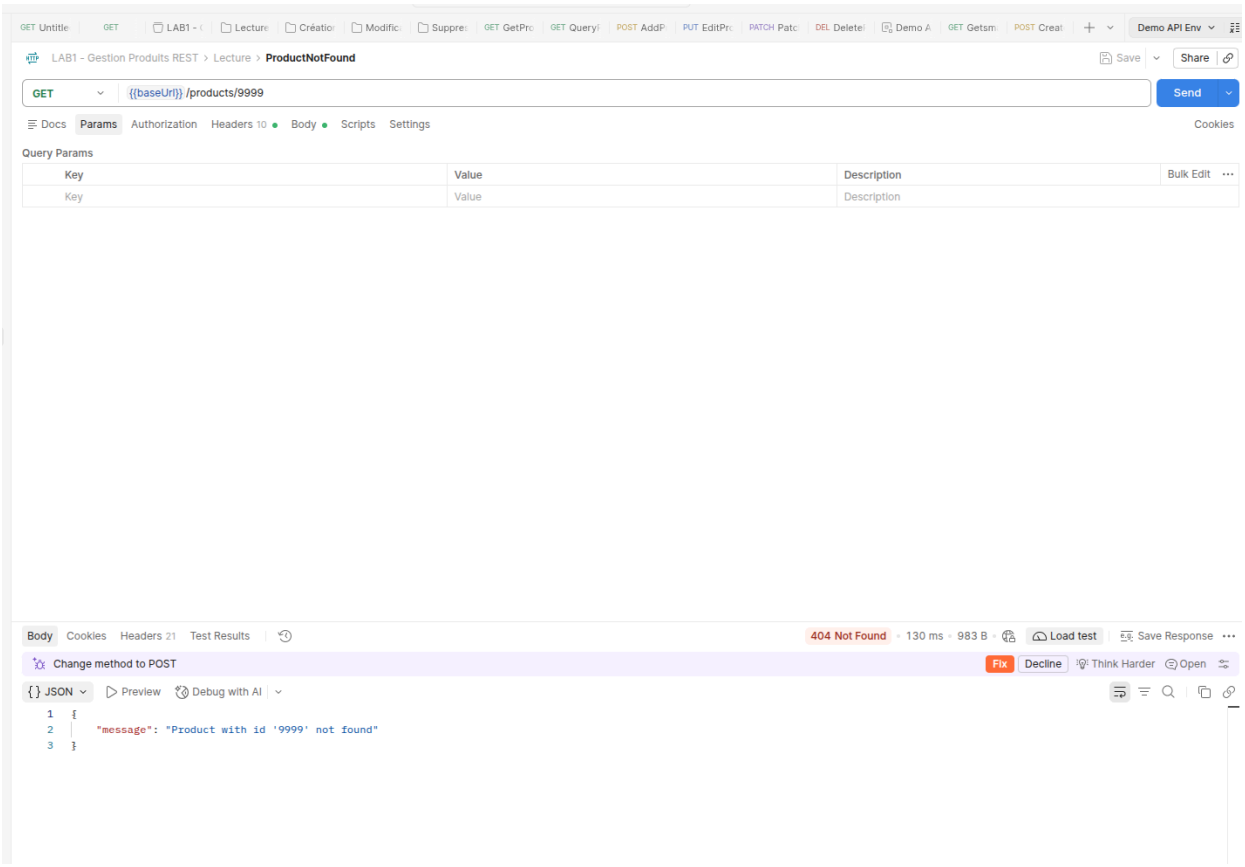
- Description : Interrogation d'un produit avec un identifiant inexistant.
- Script de Test Postman :

```javascript
pm.test("Le code de statut est 404 Not Found", function () {
    pm.response.to.have.status(404);
});
```

```
pm.test("Le corps de la réponse contient un message d'erreur clair", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('message');
    pm.expect(jsonData.message).to.include("not found");
});
```

- Structure de Réponse Obtenue :

```
JSON
{
  "message": "Product with id '99999' not found"
}
```



B. Validation du Cas d'Erreur 400 Bad Request

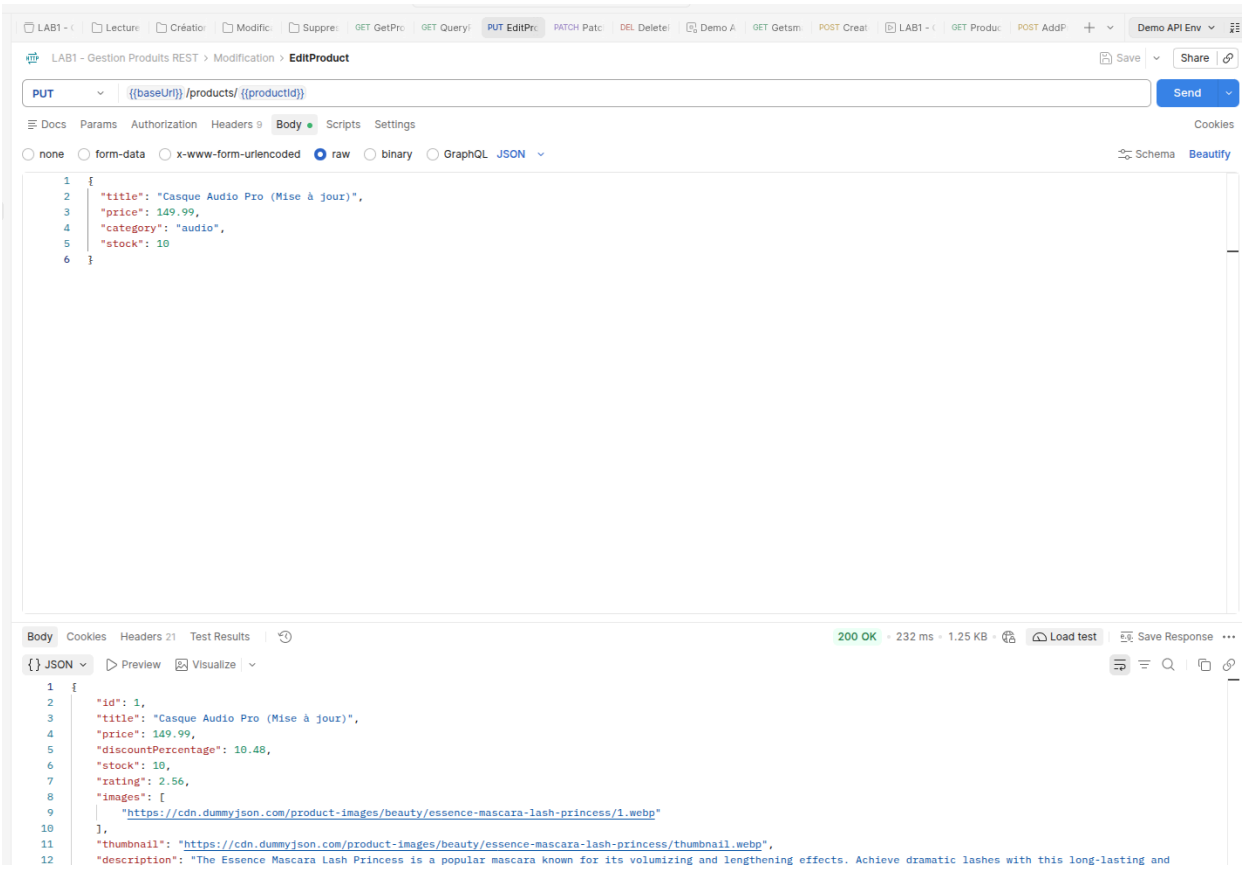
- Description : Soumission d'un payload JSON non valide (ex: champ price contenant une chaîne de caractères "abc" au lieu d'un nombre).
- Script de Test Postman :

```javascript
pm.test("Le code de statut est 400 Bad Request", function () {
    pm.response.to.have.status(400);
});

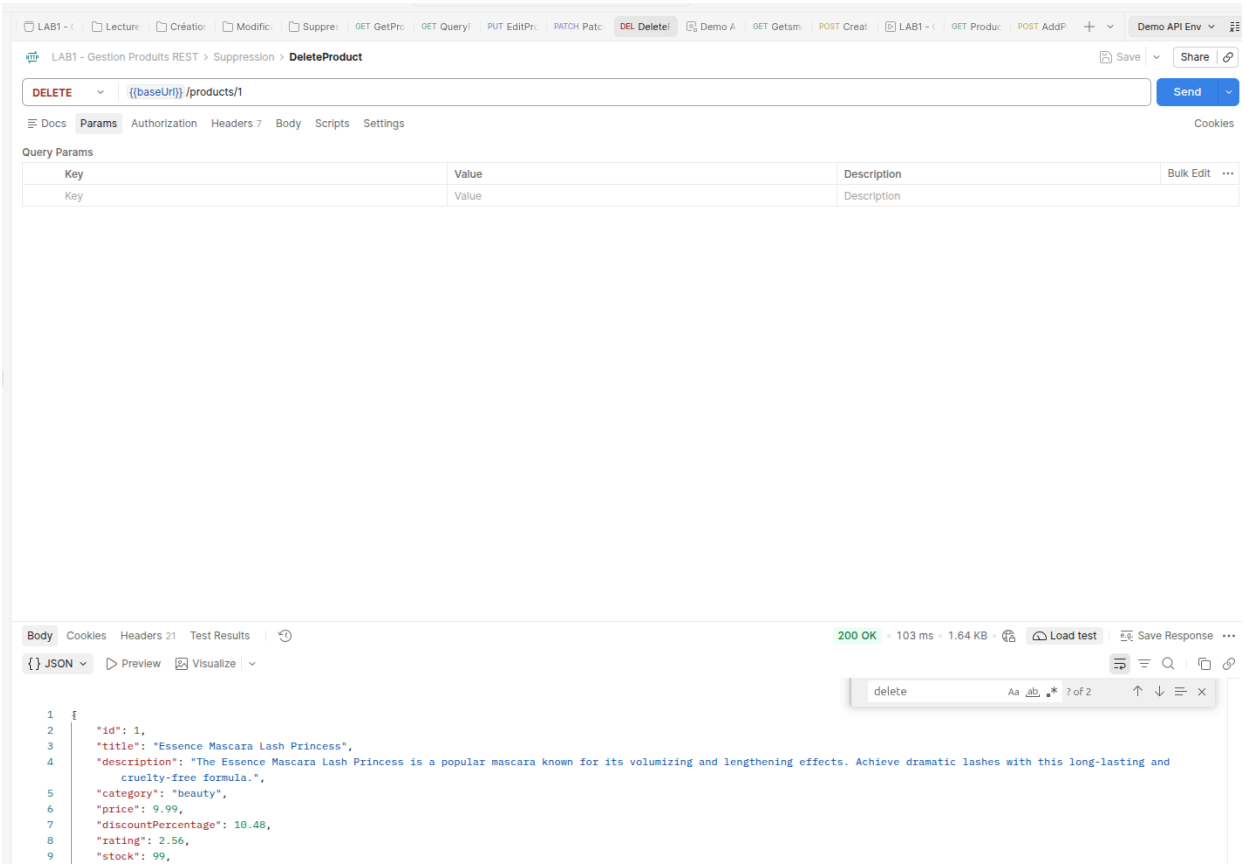
pm.test("La réponse indique une erreur de validation ou de format", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.satisfy(function(body) {
        return body.message !== undefined || body.error !== undefined;
    });
});
```

3.3. Dossier : Modification et Suppression

- EditProduct (PUT) et PatchProduct (PATCH) : Mises à jour partielles et totales du produit via {{productId}}.



- DeleteProduct (DELETE {{baseUrl}}/products/1) : Validation de la suppression de ressource.



4. Conclusion

Les tests univariés exécutés démontrent l'importance de fournir des messages d'erreur explicites au format JSON accompagnant les codes d'état HTTP adéquats (400 et 404). L'intégration systématique de scripts de validation dans Postman permet de garantir la résilience de l'API et de faciliter l'automatisation en CI/CD.